# 48. Design Azure Entra Identity Platform

## 0) Requirements & Sizing

### Functional

- **Authentication**: Authenticate users, services, and workloads across VMs, Kubernetes, and external systems
- **Authorization**: Role-Based Access Control (RBAC) with fine-grained permissions
- **Managed Identities**: Passwordless authentication for Azure resources (VMs, AKS, App Services)
- **Workload Identity**: Federated credentials for Kubernetes pods and external workloads (GitHub Actions, AWS)
- **External Identities**: B2B (partners) and B2C (customers) identity federation
- **Conditional Access**: Policy-based access control (location, device, risk level)
- **Single Sign-On (SSO)**: Seamless access across applications
- **Multi-Factor Authentication (MFA)**: Additional security layer
- **Token Management**: OAuth 2.0, OIDC, SAML token issuance and validation

### Non-functional

| Metric | Target |
|---|---|
| Authentication latency | < **100ms** p99 for token validation |
| Token issuance | < **500ms** for new token acquisition |
| Availability | **99.99%** (four 9s) - identity is critical path |
| Scale | **Millions of authentications/day** per tenant |
| Global | Multi-region with <50ms to nearest endpoint |
| Security | Zero-trust, continuous verification |

### Capacity

```
Large Enterprise Scenario:
- 100,000 employees
- 10,000 VMs with managed identities
```

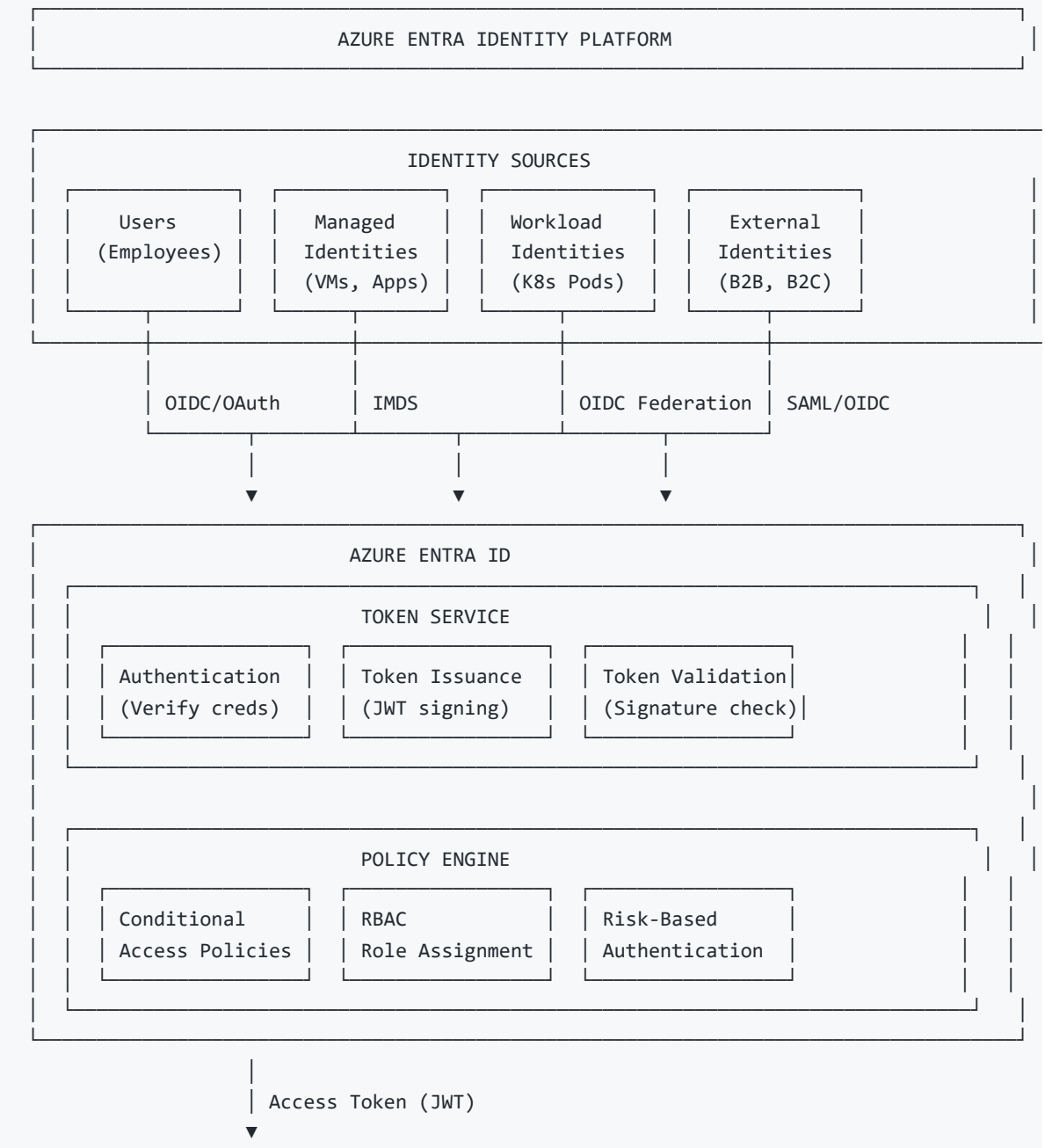
```
- 500 AKS clusters, 50,000 pods with workload identity
- 1M external partners (B2B)
- 10M customers (B2C)

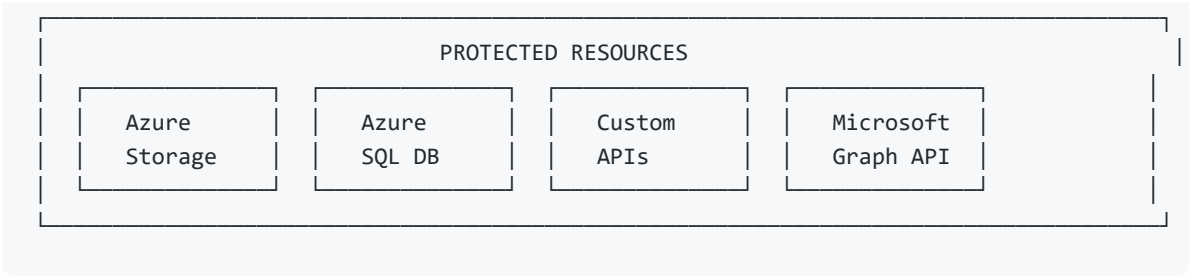
Token operations:
- Each workload: ~100 token acquisitions/day (cached, refreshed hourly)
- Peak: 10M token operations/hour
- Token size: ~2-4KB (JWT)
```

---

# 1) High-Level Architecture

## Authentication Flow Overview






| Step | From | To | Action | Sync/Async |
|---|---|---|---|---|
| 1 | Identity Source | Azure Entra | Present credentials (password, certificate, token) | SYNC |
| 2 | Azure Entra | Policy Engine | Evaluate conditional access policies | SYNC |
| 3 | Policy Engine | MFA Service | Trigger MFA if required | SYNC |
| 4 | Azure Entra | Token Service | Issue signed JWT access token | SYNC |
| 5 | Client | Resource | Present token in Authorization header | SYNC |
| 6 | Resource | Azure Entra | Validate token signature (cached keys) | SYNC |

---

# API Design

## Authentication APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/oauth2/token` | Get access token (OAuth 2.0) |
| `GET` | `/api/v1/oauth2/authorize` | Authorization endpoint |
| `POST` | `/api/v1/oauth2/revoke` | Revoke a token |
| `GET` | `/api/v1/.well-known/openid-configuration` | OpenID Connect discovery |

## User APIs

| Method | Endpoint | Description |
|---|---|---|

| | | |
|---|---|---|
| `GET` | `/api/v1/users/{userId}` | Get user profile |
| `GET` | `/api/v1/users/me` | Get current user (from token) |
| `PATCH` | `/api/v1/users/{userId}` | Update user attributes |
| `POST` | `/api/v1/users` | Create a user |
| `DELETE` | `/api/v1/users/{userId}` | Delete a user |

### Application Registration APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/applications` | Register an application |
| `GET` | `/api/v1/applications/{appId}` | Get application details |
| `PUT` | `/api/v1/applications/{appId}/permissions` | Configure API permissions |

### Token Request (Client Credentials Flow)

```
POST /api/v1/oauth2/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=app_123
&client_secret=secret_456
&scope=https://graph.microsoft.com/.default

Response: 200 OK
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "User.Read"
}
```

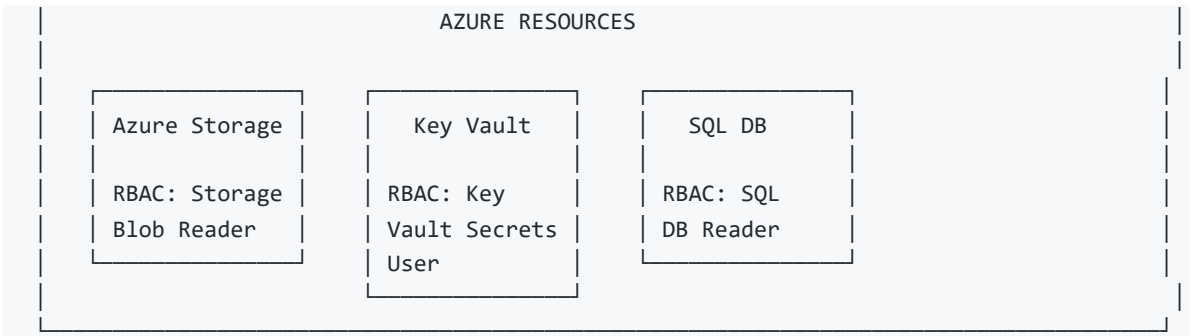
### Managed Identity APIs

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/metadata/identity/oauth2/token?resource={resource}` | Get token via managed identity (IMDS) |

## 2) Managed Identities for VMs

## Architecture






## Token Acquisition Flow

| Step | Component | Action | Details |
|---|---|---|---|
| 1 | App Code | Call `DefaultAzureCredential()` | SDK handles token acquisition |
| 2 | SDK | HTTP GET to IMDS | `http://169.254.169.254/metadata/id` `resource=https://storage.azure.com` |
| 3 | IMDS | Forward to Azure Fabric | Internal Azure infrastructure |
| 4 | Azure Entra | Issue token | Signed JWT with claims |
| 5 | SDK | Cache token | Until 5 min before expiry |
| 6 | SDK | Present to resource | `Authorization: Bearer <token>` |

## Code Example

```
// C# - Using DefaultAzureCredential (recommended)
using Azure.Identity;
using Azure.Storage.Blobs;

// Automatically uses:
// 1. Environment variables (for local dev)
// 2. Managed Identity (when deployed to Azure)
// 3. Azure CLI / Visual Studio credentials (for development)
var credential = new DefaultAzureCredential();

// Access Azure Storage - NO connection strings!
var blobClient = new BlobServiceClient(
    new Uri("https://mystorageaccount.blob.core.windows.net"),
    credential
);

// Access Key Vault - NO secrets in code!
var secretClient = new SecretClient(
    new Uri("https://mykeyvault.vault.azure.net"),
```

```
    credential
);
```

```python
# Python - Using DefaultAzureCredential
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient

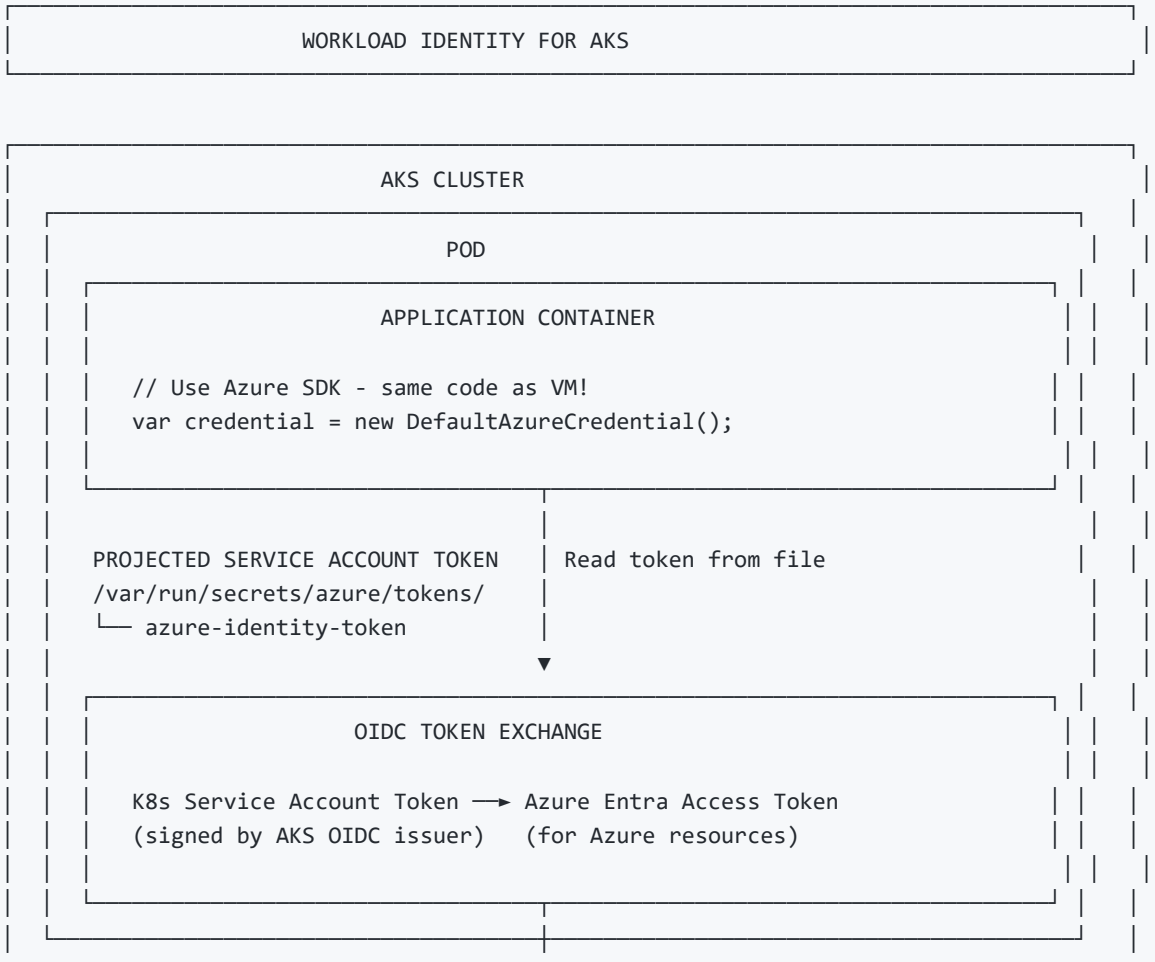
credential = DefaultAzureCredential()

# Access Azure Storage
blob_service = BlobServiceClient(
    account_url="https://mystorageaccount.blob.core.windows.net",
    credential=credential
)
```

---

# 3) Workload Identity for Kubernetes (AKS)

## Architecture






## Setup Steps

```
# 1. Create User-Assigned Managed Identity
az identity create \
  --name "my-workload-identity" \
```

```
  --resource-group "my-rg" \
  --location "eastus"

# 2. Create Federated Credential
az identity federated-credential create \
  --name "aks-federation" \
  --identity-name "my-workload-identity" \
  --resource-group "my-rg" \
  --issuer "https://oidc.prod-aks.azure.com/{tenant-id}/{cluster-id}" \
  --subject "system:serviceaccount:my-namespace:my-service-account" \
  --audience "api://AzureADTokenExchange"

# 3. Assign RBAC Role
az role assignment create \
  --assignee-object-id "<managed-identity-object-id>" \
  --role "Storage Blob Data Reader" \
  --scope "/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Storage/stor
```

```
# 4. Kubernetes ServiceAccount (with annotations)
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
  namespace: my-namespace
  annotations:
    azure.workload.identity/client-id: "<managed-identity-client-id>"
  labels:
    azure.workload.identity/use: "true"
---
# 5. Pod using the ServiceAccount
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
  labels:
    azure.workload.identity/use: "true"
spec:
  serviceAccountName: my-service-account
  containers:
  - name: app
    image: myapp:latest
    # Token automatically projected to /var/run/secrets/azure/tokens/
```

## Comparison: Pod Identity vs Workload Identity

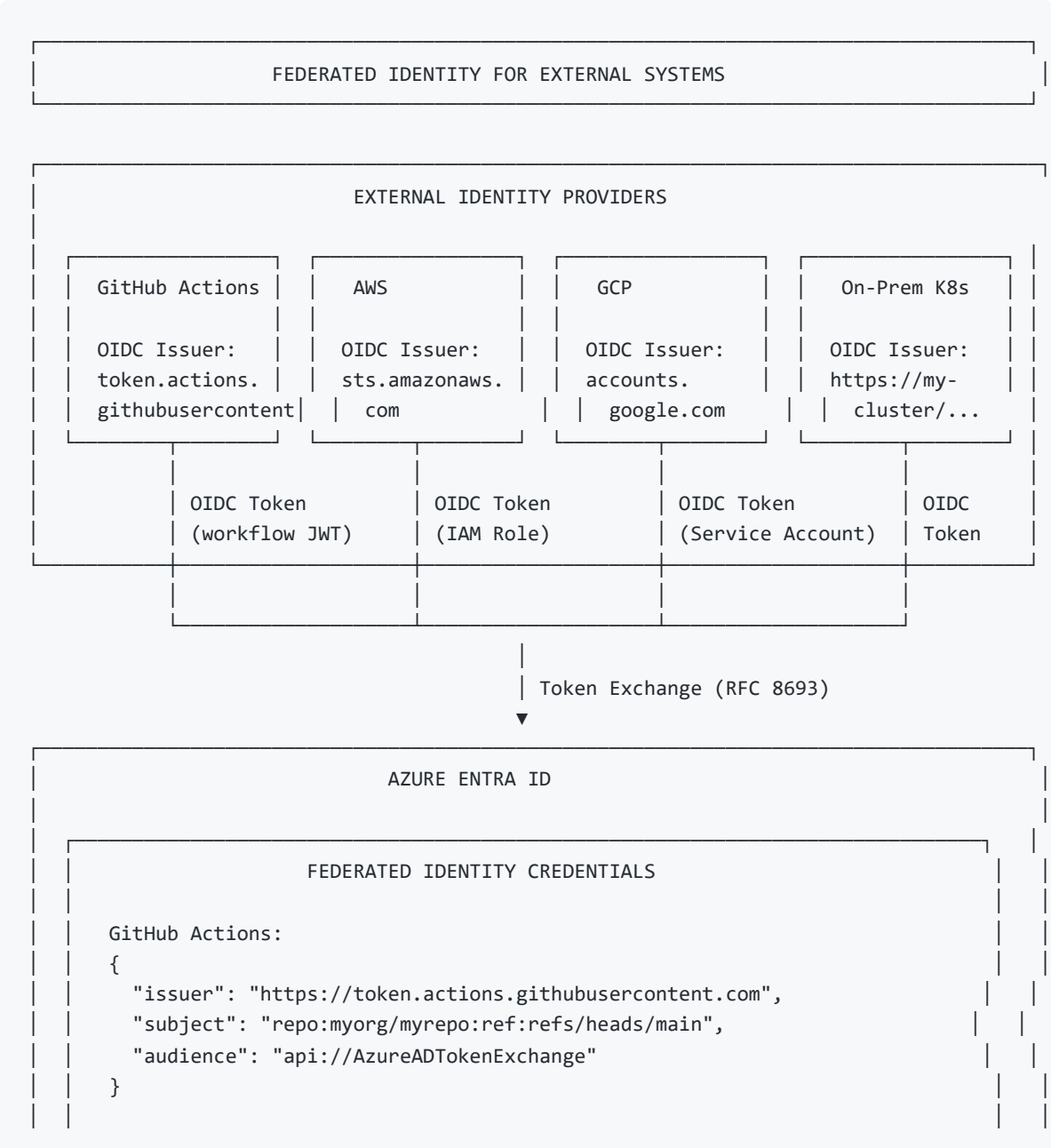
| Aspect | Pod Identity (Deprecated) | Workload Identity (Recommended) |
|---|---|---|
| Secret storage | Identity binding CRDs | No secrets in cluster |

| Token exchange | NMI DaemonSet intercepts | Direct OIDC federation |
|---|---|---|
| Security | Node-level access | Pod-level, fine-grained |
| Multi-cluster | Complex | Easy (same federation) |
| Non-AKS K8s | No | Yes (any OIDC issuer) |
| Network | Requires NMI pod | No extra pods |

---

# 4) External/Federated Identities

## Architecture for External Workloads






## GitHub Actions Example

```yaml
# .github/workflows/deploy.yml
name: Deploy to Azure
on:
  push:
    branches: [main]

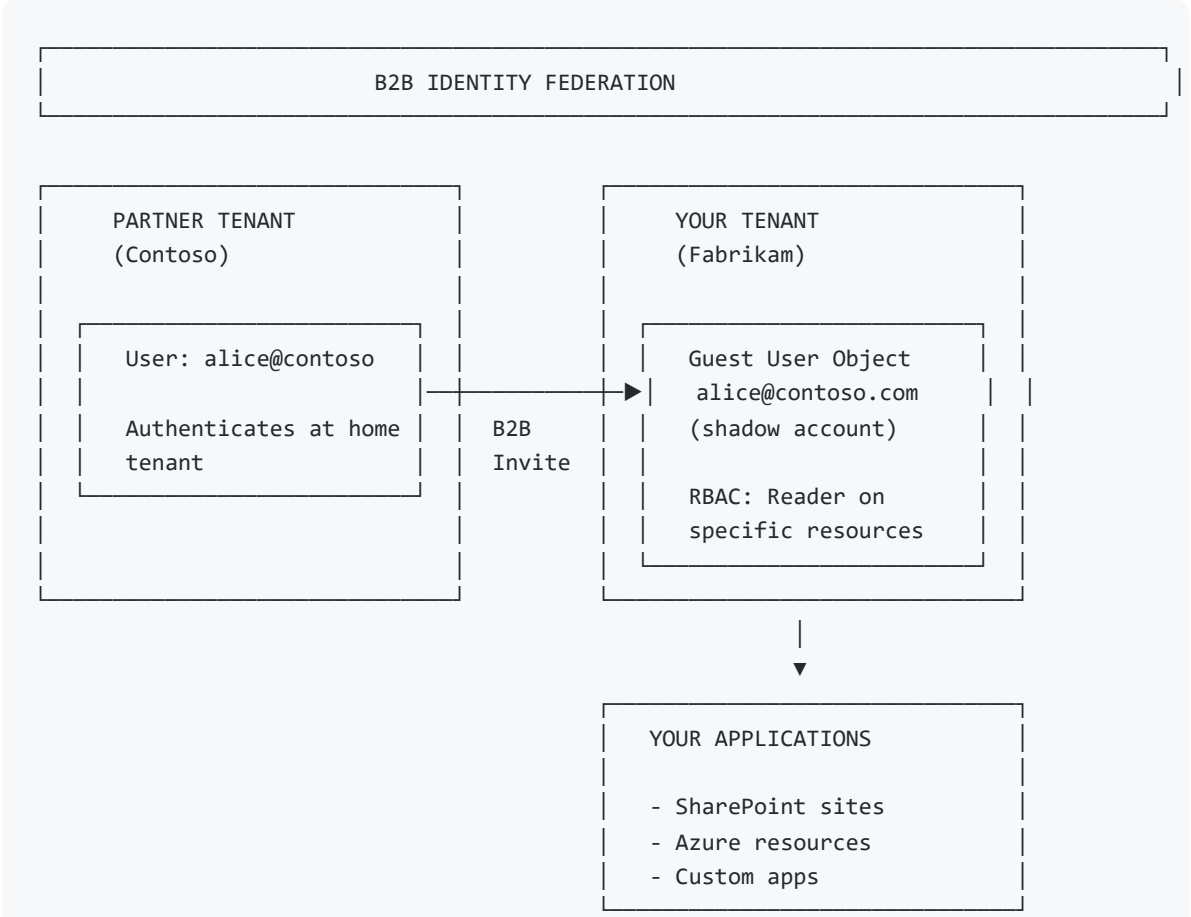
permissions:
  id-token: write   # Required for OIDC
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

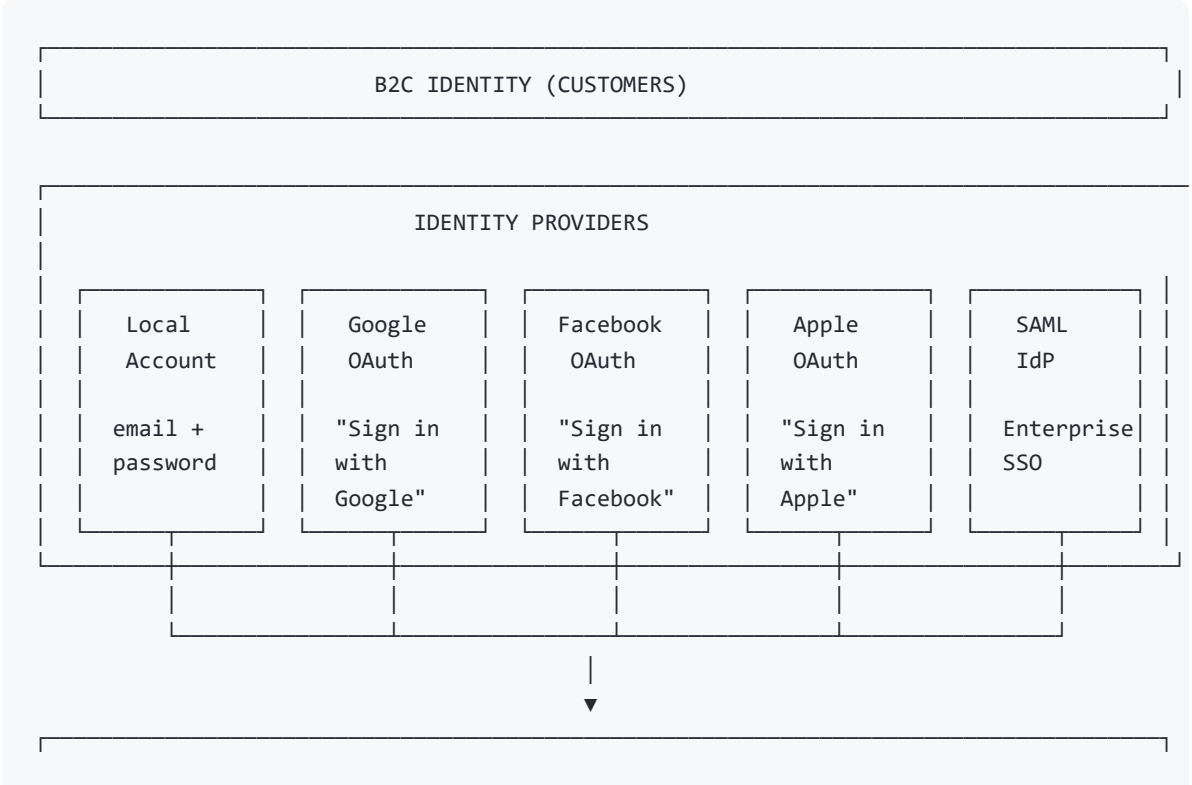
      # Login using OIDC - NO secrets stored in GitHub!
      - name: Azure Login
        uses: azure/login@v1
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

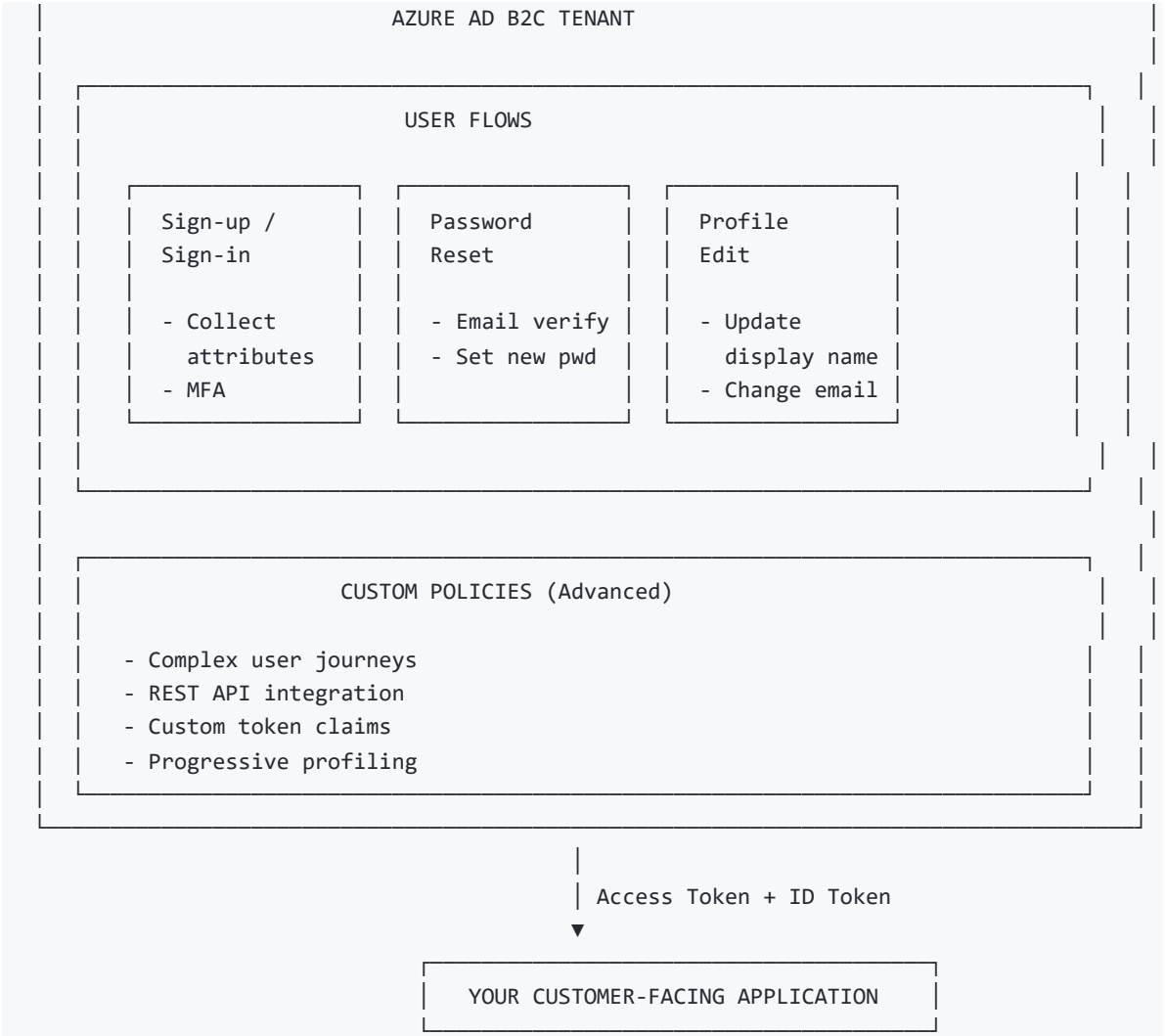
      # Deploy resources
      - name: Deploy
        run: |
          az webapp deploy --name myapp --src-path ./dist
```

## B2B (Business Partners) Federation




## B2C (Customer) Identity






# 5) Token Architecture

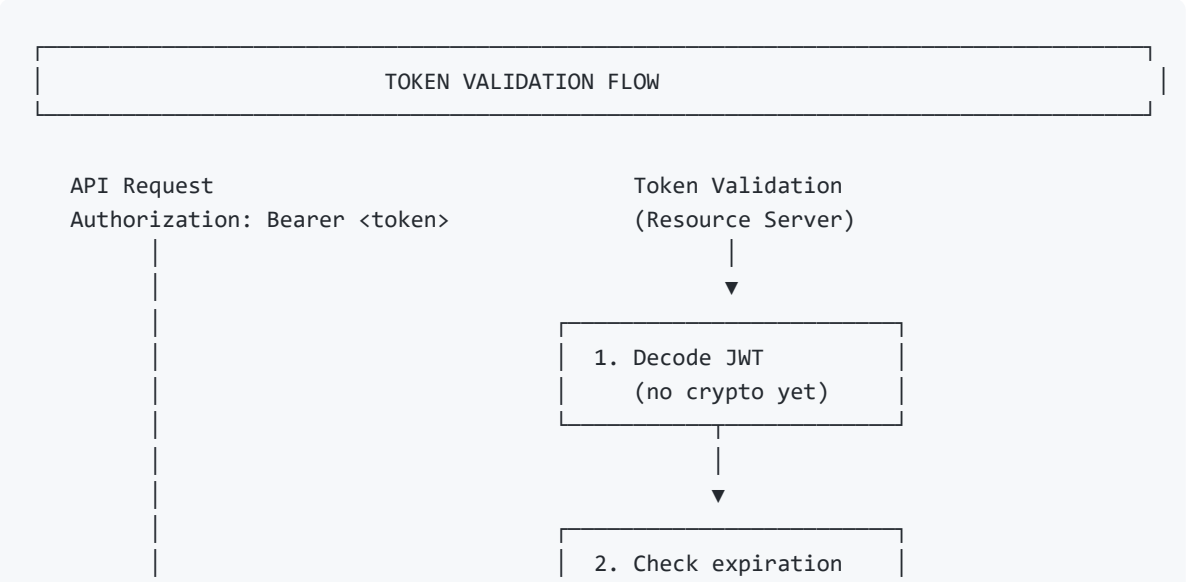
## JWT Structure

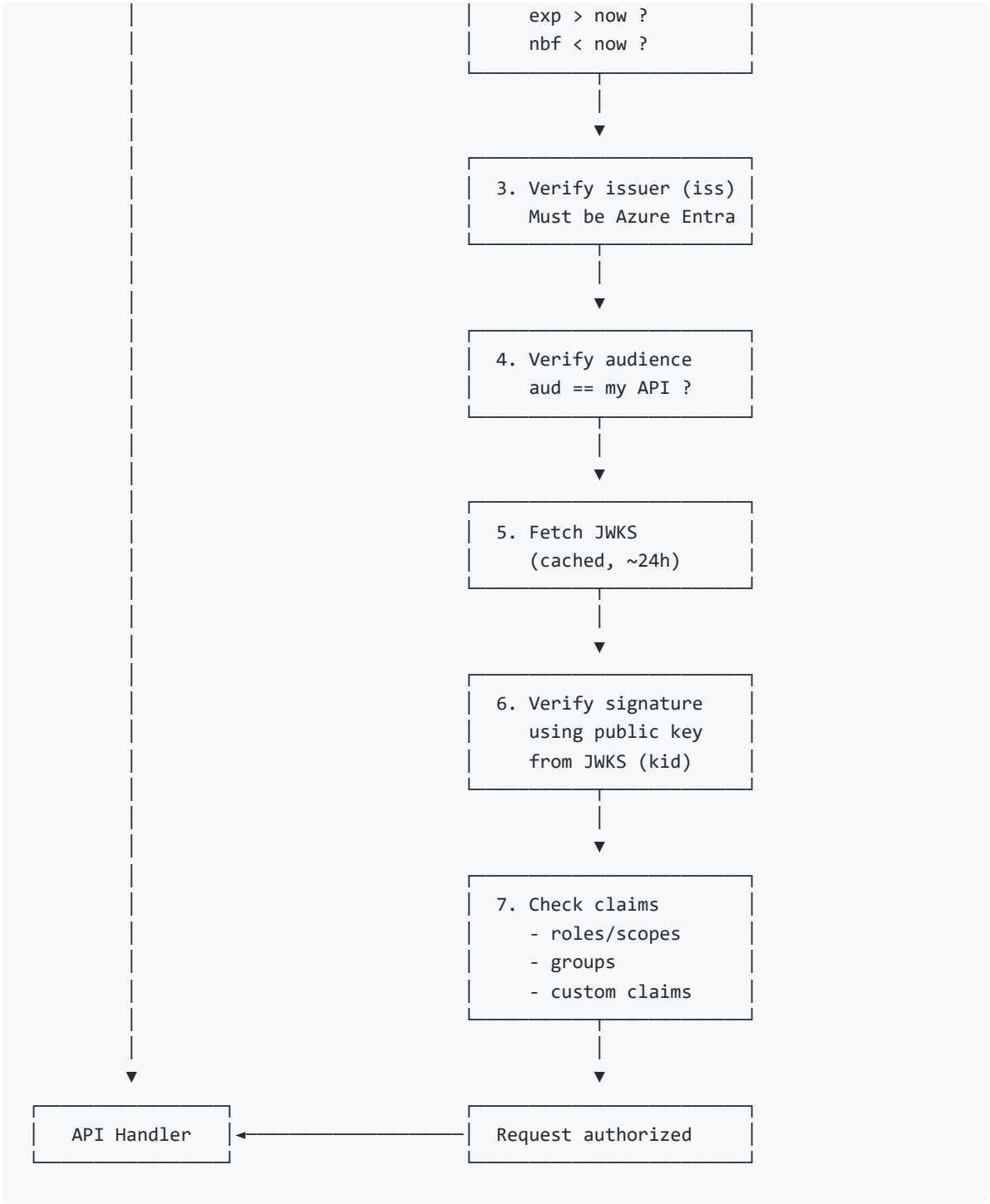


```
┌──────────────────────────────────────────────────────────────────────────────────┐
│  PAYLOAD (Claims)                                                                │
│  {                                                                               │
│    // Standard claims                                                            │
│    "iss": "https://sts.windows.net/{tenant-id}/",     // Issuer                  │
│    "sub": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Subject (user/app ID)   │
│    "aud": "https://storage.azure.com",                // Audience (target resource)│
│    "exp": 1706364000,                                  // Expiration (Unix timestamp)
│    "iat": 1706360400,                                  // Issued at               │
│    "nbf": 1706360400,                                  // Not before              │
│                                                                                  │
│    // Azure-specific claims                                                      │
│    "tid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Tenant ID               │
│    "oid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Object ID               │
│    "azp": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",     // Authorized party (client) │
│    "scp": "user_impersonation",                        // Scopes (delegated)      │
│    "roles": ["Storage.BlobReader"],                    // App roles              │
│                                                                                  │
│    // Optional claims                                                            │
│    "name": "John Doe",                                                           │
│    "email": "john@contoso.com",                                                  │
│    "groups": ["group-id-1", "group-id-2"],            // Group memberships       │
│    "xms_cc": ["CP1"]                                   // Client capabilities     │
│  }                                                                               │
└──────────────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────────┐
│  SIGNATURE                                                                       │
│                                                                                  │
│  RS256(Base64URL(header) + "." + Base64URL(payload), private_key)                │
│                                                                                  │
│  - Signed by Azure Entra's private key                                           │
│  - Verified using public key from JWKS endpoint                                  │
│  - JWKS URL: https://login.microsoftonline.com/{tenant}/discovery/v2.0/keys      │
└──────────────────────────────────────────────────────────────────────────────────┘
```

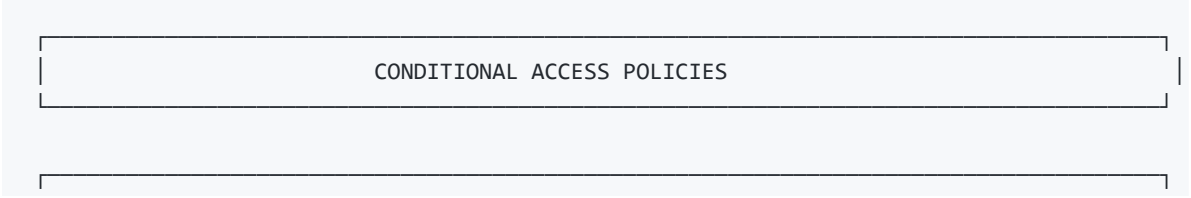
## Token Validation Flow

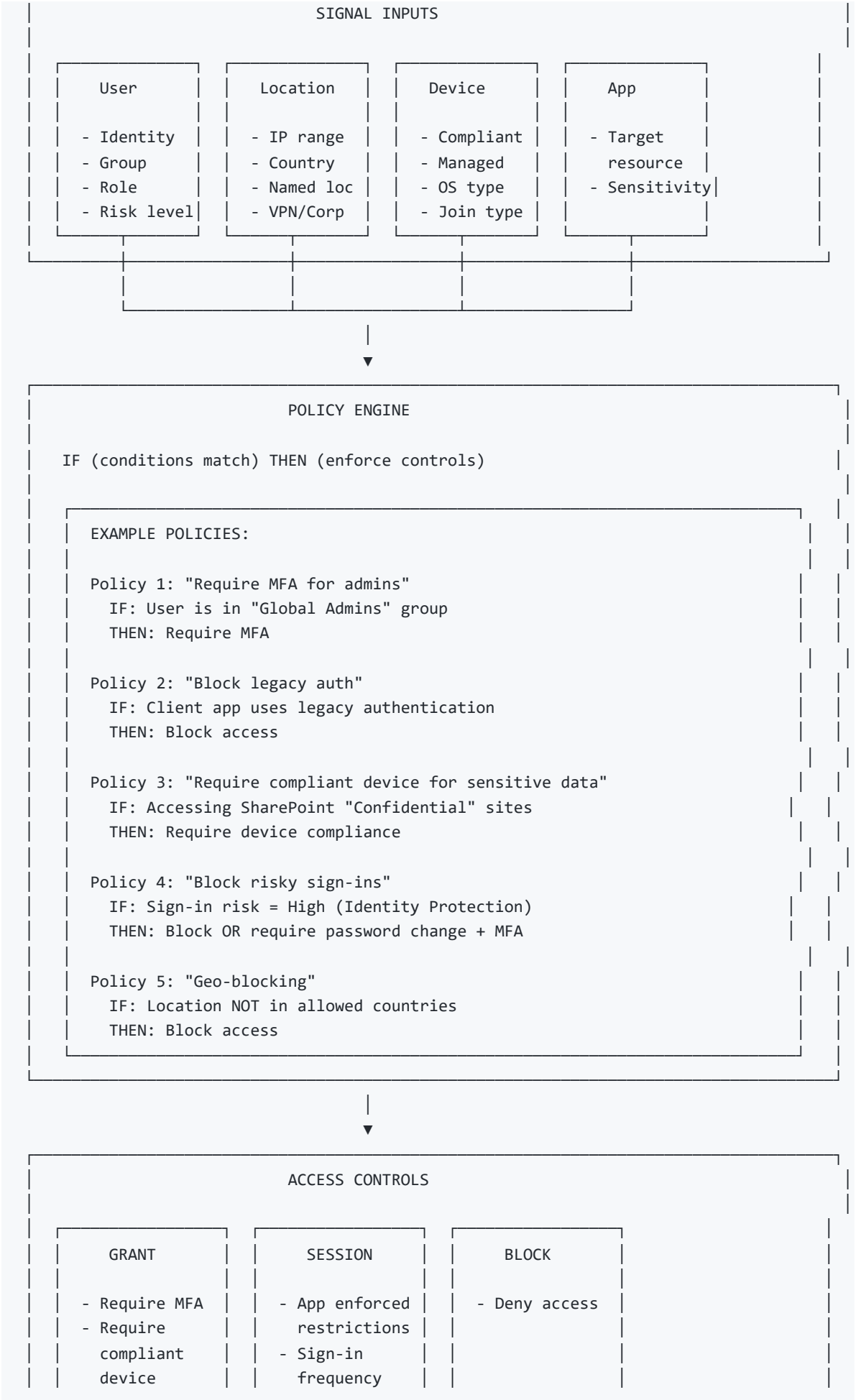





# 6) Conditional Access & Zero Trust

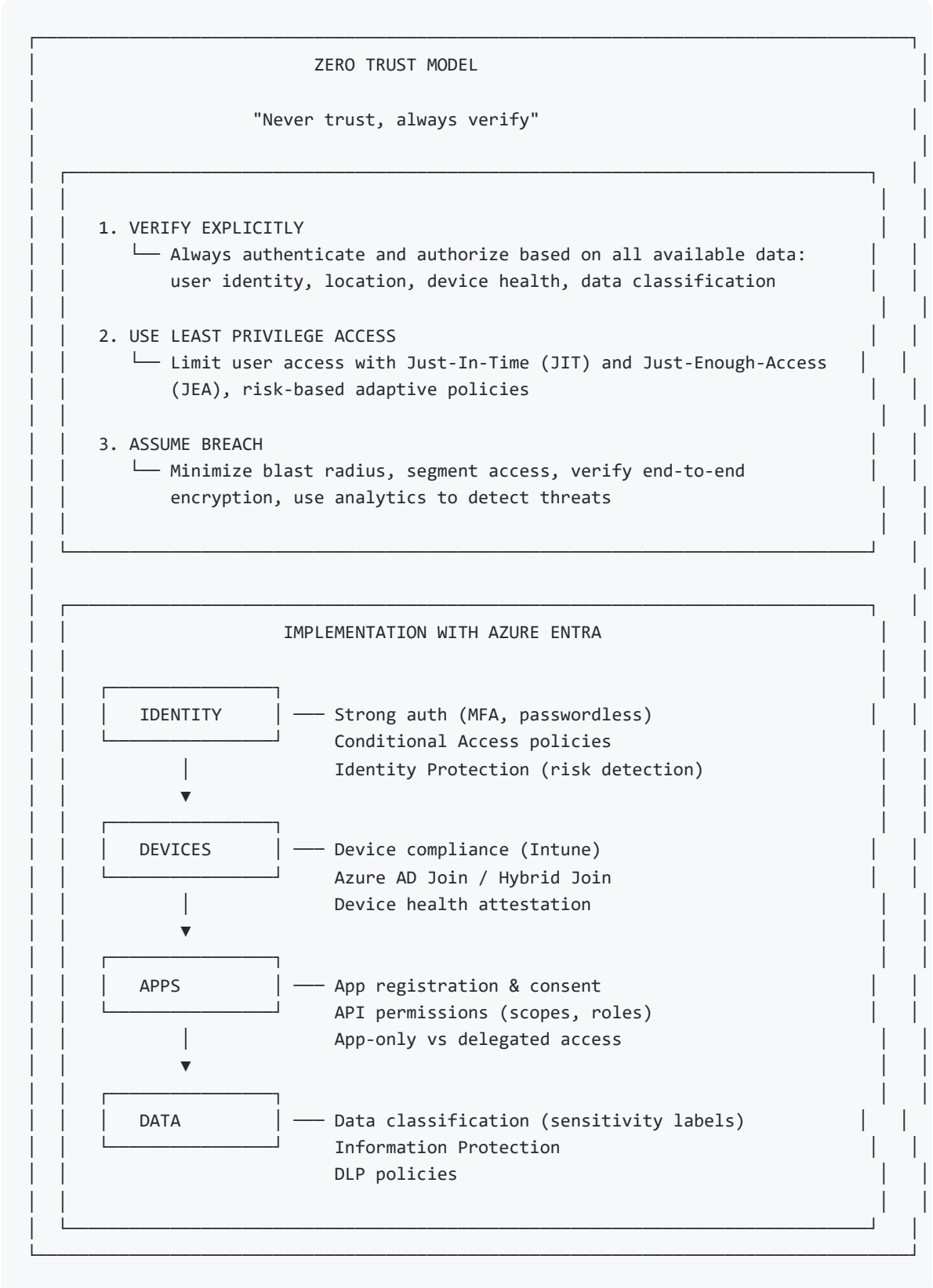
## Policy Architecture




SIGNAL INPUTS
User
- Identity
- Group
- Role
- Risk level
Location
- IP range
- Country
- Named loc
- VPN/Corp
Device
- Compliant
- Managed
- OS type
- Join type
App
- Target resource
- Sensitivity
POLICY ENGINE
IF (conditions match) THEN (enforce controls)
EXAMPLE POLICIES:
Policy 1: "Require MFA for admins"
IF: User is in "Global Admins" group
THEN: Require MFA
Policy 2: "Block legacy auth"
IF: Client app uses legacy authentication
THEN: Block access
Policy 3: "Require compliant device for sensitive data"
IF: Accessing SharePoint "Confidential" sites
THEN: Require device compliance
Policy 4: "Block risky sign-ins"
IF: Sign-in risk = High (Identity Protection)
THEN: Block OR require password change + MFA
Policy 5: "Geo-blocking"
IF: Location NOT in allowed countries
THEN: Block access
ACCESS CONTROLS
GRANT
- Require MFA
- Require compliant device
SESSION
- App enforced restrictions
- Sign-in frequency
BLOCK
- Deny access



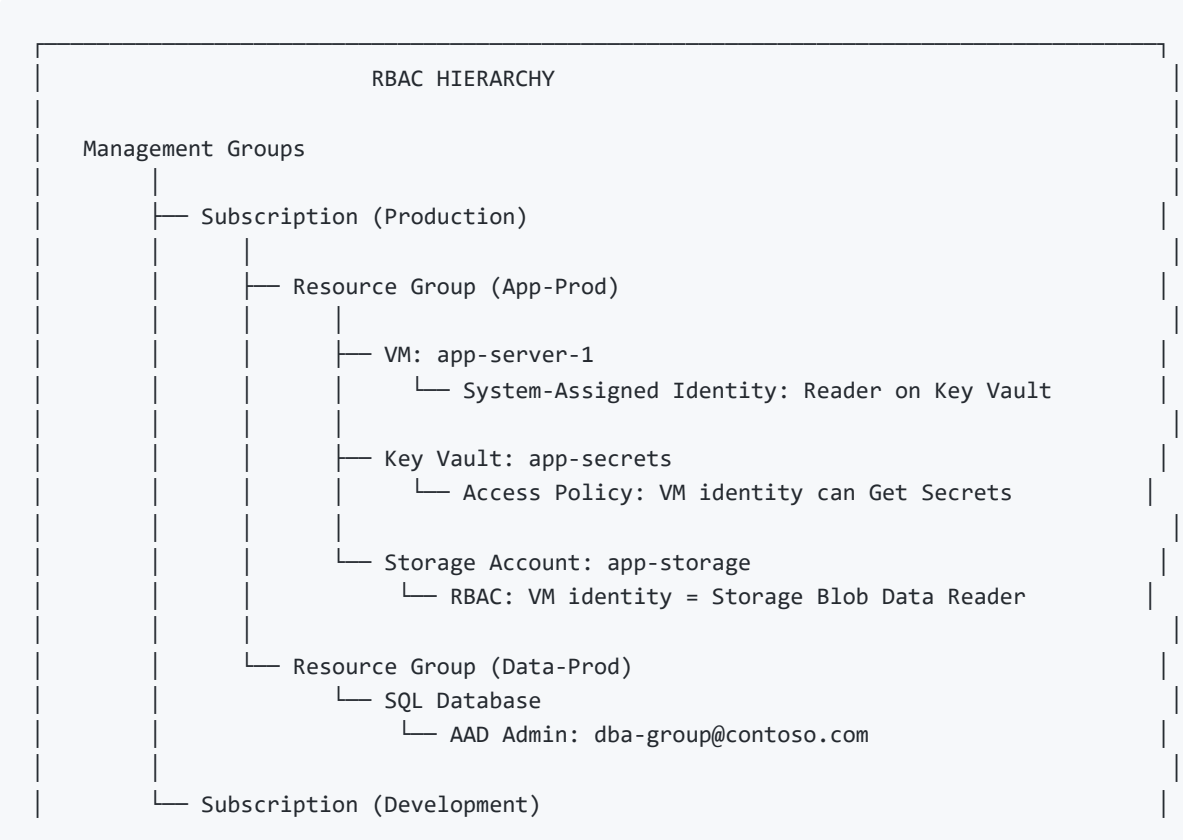

## Zero Trust Principles



## 7) Security & Governance

| Concern | Solution |
|---|---|
| Credential theft | Managed identities (no passwords), passwordless auth (FIDO2, Windows Hello) |
| Token theft | Short-lived tokens (1h), Continuous Access Evaluation (CAE), token binding |
| Lateral movement | Conditional Access, PIM for just-in-time admin access |
| Compromised identity | Identity Protection (risk detection), automated remediation |
| Over-privileged access | RBAC with least privilege, Access Reviews, PIM |
| Audit trail | Sign-in logs, Audit logs, Azure Monitor integration |
| External threats | Geo-blocking, IP restrictions, risky sign-in policies |
| Legacy protocols | Block legacy authentication (IMAP, POP3, SMTP) |

### RBAC Best Practices

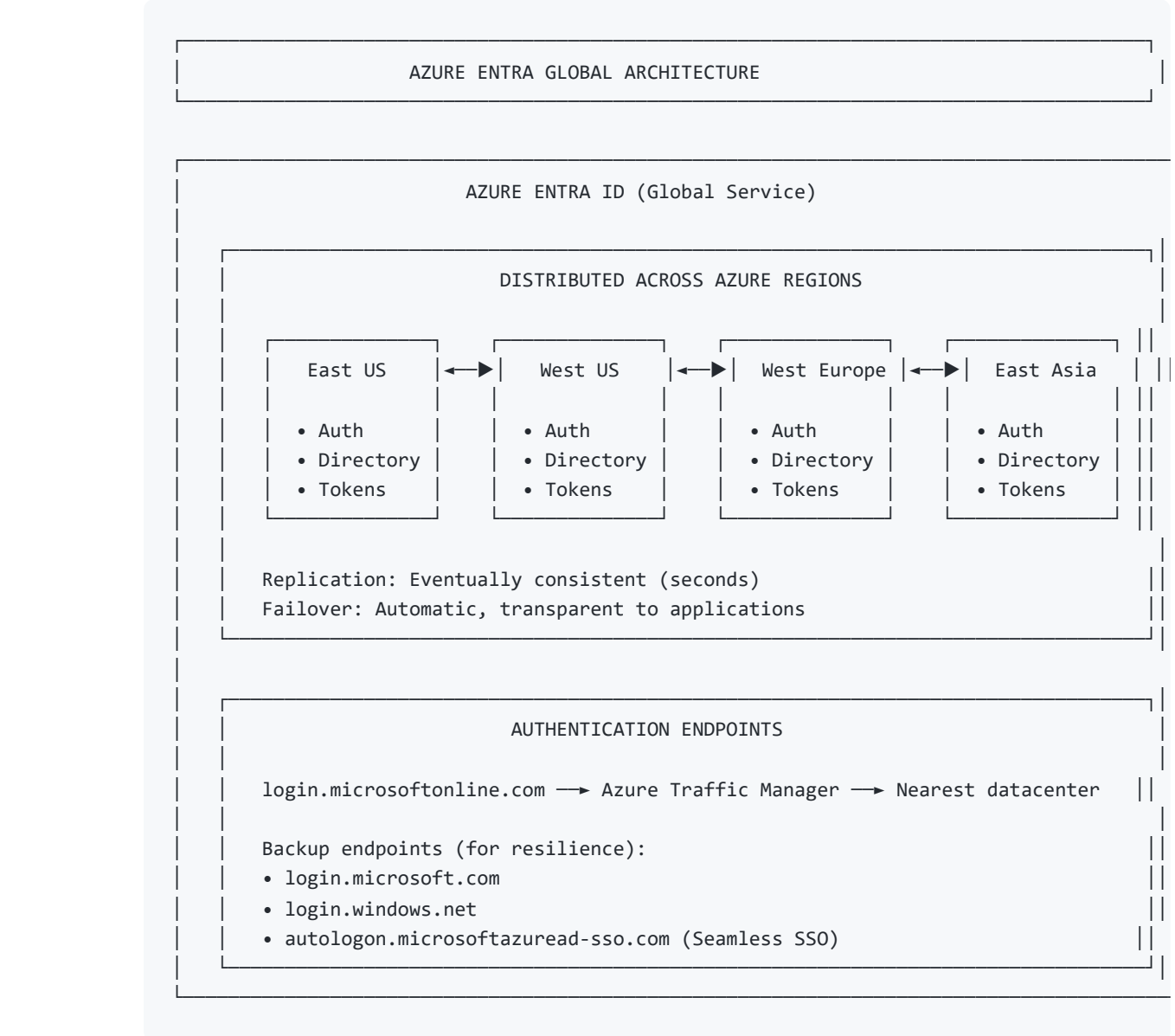


```
│               └── (Developers have Contributor, not in Prod)                  │
│                                                                               │
└───────────────────────────────────────────────────────────────────────────────┘

PRINCIPLE: Assign at lowest scope necessary, use groups not individuals
```

---

## 8) Multi-Region & High Availability




### Resilience Best Practices

| Scenario | Mitigation |
|---|---|
| Azure Entra partial outage | Token caching (MSAL default: 1h), fallback to cached credentials |

| | |
|---|---|
| Network connectivity | Multiple auth endpoints, retry with exponential backoff |
| Regional failure | Azure Entra auto-routes to healthy region |
| Token validation offline | Cache JWKS keys locally (24h TTL) |

---

## 9) Failure Modes

| Failure | Impact | Mitigation |
|---|---|---|
| Azure Entra outage | Cannot acquire new tokens | Token caching, long-lived refresh tokens (90 days) |
| IMDS unavailable | VMs can't get tokens | Retry logic, circuit breaker, fallback to service principal |
| AKS OIDC issuer down | Pods can't exchange tokens | Cache tokens aggressively, graceful degradation |
| Network partition | Can't reach auth endpoints | Validate tokens locally with cached JWKS |
| Token expired | 401 Unauthorized | Proactive token refresh (5 min before expiry) |
| Credential rotation | Brief auth failures | Blue-green credential rotation, overlap validity period |

### Token Caching Strategy

```
// MSAL handles caching automatically, but you can configure it
var app = ConfidentialClientApplicationBuilder
    .Create(clientId)
    .WithAuthority(authority)
    .WithClientSecret(clientSecret)
    .Build();

// In-memory cache (default) or distributed cache (Redis)
app.AddDistributedTokenCache(services =>
{
    services.AddDistributedMemoryCache(); // Or Redis
    services.Configure<MsalDistributedTokenCacheAdapterOptions>(o =>
    {
        o.AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24);
    });
});
```

```
// Acquire token (uses cache automatically)
var result = await app.AcquireTokenForClient(scopes)
    .ExecuteAsync();
```

---

## 10) Cost Estimates

| Component | Tier | Monthly Cost (Est.) |
|---|---|---|
| **Azure Entra ID P1** | Per user/month | ~$6/user |
| **Azure Entra ID P2** | Per user/month (includes P1 + Identity Protection, PIM) | ~$9/user |
| **B2C** | First 50K MAU free, then $0.00325/MAU | Variable |
| **Managed Identities** | Free | $0 |
| **Workload Identity** | Free | $0 |
| **Conditional Access** | Included in P1/P2 | $0 (with P1/P2) |

**Example: 10,000 employee enterprise**

- Azure Entra ID P2: 10,000 × $9 = **$90,000/year**
- Managed Identities: Free
- B2C (1M customers): ~$3,250/month = **$39,000/year**
- **Total: ~$129,000/year**

---

## 11) Trade-offs Discussion

| Decision | Option A | Option B | Choice | Reasoning |
|---|---|---|---|---|
| VM Identity | System-assigned | User-assigned | **Depends** | System for single VM; User for VMSS/shared |
| K8s auth | Pod Identity | Workload Identity | **Workload** | No secrets in cluster, future-proof |

| | | | | |
|---|---|---|---|---|
| External workloads | Service Principal + Secret | Federated Credential | **Federated** | No secrets to manage/rotate |
| B2B | Guest accounts | External identities | **Guest** | Full integration with your tenant |
| B2C | Azure AD B2C | 3rd party (Auth0) | **B2C** | Native Azure integration, cost |
| Token lifetime | Short (1h) | Long (24h) | **Short** | Security; use refresh tokens for UX |

---

## 12) 60-Second Explanation

> "Azure Entra ID is Microsoft's cloud identity platform implementing **Zero Trust**: never trust, always verify.
>
> For **Azure VMs**, I use **Managed Identities** - the VM gets an identity automatically, acquires tokens via the Instance Metadata Service at 169.254.169.254, and authenticates to Azure resources without any credentials in code.
>
> For **Kubernetes**, **Workload Identity** federates the AKS OIDC issuer with Azure Entra. Pods get service account tokens that are exchanged for Azure AD tokens - again, no secrets in the cluster.
>
> For **external workloads** like GitHub Actions or AWS, I create **Federated Identity Credentials** that trust external OIDC issuers. The external system presents its token, Azure Entra validates it against the trusted issuer, and issues an Azure token.
>
> For **partners and customers**, **B2B** invites external users as guests (they authenticate at their home tenant), and **B2C** handles consumer identities with social logins.
>
> Everything flows through **Conditional Access** policies that evaluate signals (user, device, location, risk) and enforce controls (MFA, compliance). Combined with short-lived tokens and continuous access evaluation, this implements defense-in-depth."

---

## Technologies Deep Dive

| Technology | Role | Reference |
|---|---|---|
| OAuth 2.0 / OIDC | Token-based authentication protocols | Tech Guide |

| | | |
|---|---|---|
| JWT | Token format for claims | Tech Guide |
| SAML | Legacy federation protocol | Tech Guide |
| FIDO2 | Passwordless authentication | Tech Guide |

---

## Appendix: DefaultAzureCredential Chain

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    DefaultAzureCredential - Credential Chain                 │
└──────────────────────────────────────────────────────────────────────────────┘

The SDK tries each credential source in order until one succeeds:

┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  1. ENVIRONMENT VARIABLES                                                    │
│     └── AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID                │
│     └── Used in: CI/CD pipelines, containers                                 │
│                                                                              │
│  2. WORKLOAD IDENTITY (AKS)                                                  │
│     └── AZURE_FEDERATED_TOKEN_FILE, AZURE_CLIENT_ID, AZURE_TENANT_ID         │
│     └── Used in: Kubernetes pods with workload identity                      │
│                                                                              │
│  3. MANAGED IDENTITY                                                         │
│     └── System-assigned or user-assigned                                     │
│     └── Used in: VMs, App Service, Functions, AKS                            │
│                                                                              │
│  4. AZURE CLI                                                                │
│     └── az login credentials                                                 │
│     └── Used in: Local development                                           │
│                                                                              │
│  5. AZURE POWERSHELL                                                         │
│     └── Connect-AzAccount credentials                                        │
│     └── Used in: Local development                                           │
│                                                                              │
│  6. VISUAL STUDIO                                                            │
│     └── Logged-in Azure account                                              │
│     └── Used in: Local development in VS                                     │
│                                                                              │
│  7. VISUAL STUDIO CODE                                                       │
│     └── Azure Account extension                                              │
│     └── Used in: Local development in VS Code                                │
│                                                                              │
│  8. INTERACTIVE BROWSER                                                      │
│     └── Opens browser for login (if enabled)                                 │
│     └── Used in: Local development, interactive apps                         │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

// Code works everywhere without changes:
```

```
var credential = new DefaultAzureCredential();

// Local dev: Uses Azure CLI
// VM: Uses Managed Identity
// AKS: Uses Workload Identity
// CI/CD: Uses Environment Variables
```

---

# Appendix: Common Scenarios

## Scenario 1: VM accessing Key Vault

```
# 1. Enable system-assigned managed identity on VM
az vm identity assign --name myVM --resource-group myRG

# 2. Grant access to Key Vault
az keyvault set-policy --name myKeyVault \
  --object-id $(az vm show --name myVM --resource-group myRG --query identity.princi
  --secret-permissions get list

# 3. In VM, app code just works
```

## Scenario 2: AKS pod accessing Storage

```
# 1. Create user-assigned managed identity
az identity create --name myWorkloadId --resource-group myRG

# 2. Create federated credential
az identity federated-credential create \
  --name k8s-federation \
  --identity-name myWorkloadId \
  --resource-group myRG \
  --issuer $(az aks show -n myCluster -g myRG --query "oidcIssuerProfile.issuerUrl"
  --subject system:serviceaccount:default:my-service-account

# 3. Assign RBAC role
az role assignment create \
  --assignee $(az identity show -n myWorkloadId -g myRG --query principalId -o tsv)
  --role "Storage Blob Data Reader" \
  --scope /subscriptions/.../storageAccounts/mystorageaccount

# 4. Annotate ServiceAccount and deploy
```

## Scenario 3: GitHub Actions deploying to Azure

```
# 1. Create app registration
az ad app create --display-name "github-actions-deploy"

# 2. Create federated credential
az ad app federated-credential create --id <app-id> --parameters '{
  "name": "github-main-branch",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:myorg/myrepo:ref:refs/heads/main",
  "audiences": ["api://AzureADTokenExchange"]
}'

# 3. Assign RBAC role to service principal
az role assignment create \
  --assignee <app-id> \
  --role "Contributor" \
  --scope /subscriptions/.../resourceGroups/myRG

# 4. In GitHub Actions, use azure/login with OIDC
```